Fecha: 03 de noviembrede 2026
versión: 1.0



fluX One

Acta de Reunión

Comite flux

Proyecto	Infraestructura de producción en AWS — Flux y Carpe by Flux
Tipo de sesión	Revisión interna de la propuesta de arquitectura presentada por Advanced
Fecha	3 de noviembre de 2026 (inferida por el contexto de la conversación)
Modalidad	Videollamada, con una llamada breve a Ari, de Advanced, durante la sesión

1. Objetivo de la sesión

Presentar la propuesta de Santiago para pasar de microservicios a monolitos modulares con menor costo de infraestructura y mejor seguridad, y decidir cómo se separan Flux y Carpe, el CI/CD y la base de datos.

2. Propuesta presentada

- Pasar de microservicios a monolitos modulares: el volumen actual no justifica la capacidad instalada, que ya cuesta, y los microservicios consumen mucha memoria RAM por mantener vivas las JVM de Java. Sigue siendo escalable hacia el futuro. Santiago estima que el cambio de código es de poco esfuerzo.
- Sacar los frontends de las EC2 a CloudFront con firewall: unos 15 dólares por aplicación en producción, con límites de 10 millones de peticiones y 50 TB de transferencia. Después de CloudFront entran a un balanceador de carga interno y al core de cada aplicación.
- Un servicio de integración (o proxy) para cargas batch, descargas de archivos, reportes y notificaciones a clientes y terceros, de forma asíncrona. Se comunica con el core por SQS con DLQ y registra en la base de datos qué se entregó. SQS ofrece un millón de mensajes mensuales sin costo.
- Todas las salidas a terceros pasan por un NAT Gateway, que da IP fija, algo que varios proveedores y clientes piden para verificar el origen de las notificaciones.
- Llevar secretos y llaves, hoy expuestos en el código, a Secrets Manager con KMS. Santiago lo calificó como un riesgo alto, también ante un posible caso de fraude.
- Buckets S3 para archivos de auditoría. El costo fuerte está en las máquinas y las bases de datos.
- Tres ambientes (desarrollo, calidad y producción) con un costo de cerca de 500 dólares al mes, que se evalúa bajar. Desarrollo y calidad se encienden de 7 a. m. a 6 p. m. y se apagan solos; se puede hacer excepción para trabajo urgente.
- Base de datos: Carpe usa MariaDB y Flux, Postgres. Santiago propuso pasar Carpe a Postgres y alojar ambas bases en una sola RDS, con un ahorro cercano a 150 dólares. Aclaró que el problema actual no es de CPU sino del número de conexiones de tantos microservicios, por lo que crecer la máquina no lo resuelve.
- CI/CD: en lugar de un servidor dedicado de Jenkins (con mantenimiento y soporte), usar CodePipeline y CodeBuild de AWS, que prestan la máquina solo durante la construcción. Estima menos de 3.000 minutos al mes, con compilaciones de Java de 10 a 25 minutos.
- Control de versiones en CodeCommit, por el control de accesos centralizado, el costo y la integración con AWS.



3. Discusión y decisiones

3.1 Separación de Flux y Carpe

Carlos pidió argumentar por qué cambiar de MariaDB a Postgres más allá del precio y planteó que los dos productos tienen presupuestos distintos. Carpe podría ser reclamado o entregado en algún momento, y ante una revisión no se justificaría que corran en la misma máquina. Además, Flux es una suite de aplicaciones y Carpe es una sola, por lo que una caída compartida los hundiría a ambos.

- Santiago indicó que técnicamente no hay motivo para cambiar la base de datos salvo el costo, y que las bases comparten hardware pero siguen siendo idependientes. Reconoció el riesgo de que una caída de la máquina afecte a ambos, aunque la disponibilidad prevista es de 99,5 % anual.
- Alejo: compartir recursos no sería impedimento para una entrega, porque se cede el código y la base de datos y no la infraestructura. Si se quieren costos independientes, hay que independizar todo (balanceador, WAF y CloudFront incluidos). Los recursos se etiquetan para que la factura discrimine cuánto es de cada uno.
- Pablo y el participante E: prefieren que sean independientes, por tema de producto, de negocio y de riesgo. Carlos añadió que su criterio de control de riesgos es evitar que se materialice, y que el único riesgo debería ser que se caiga AWS.
- Santiago explicó que la separación implica duplicar el balanceador, el NAT Gateway y las máquinas, con unos 30 dólares más de costo. Los frontends y CloudFront ya están separados.
- Decisión: producción de Flux y de Carpe queda totalmente independiente (máquinas, balanceador y salida por NAT Gateway propios). Desarrollo y calidad comparten máquinas. Se mantiene una sola cuenta de AWS, con un etiquetado riguroso. Se pide una nueva propuesta con las dos arquitecturas independientes y los tres ambientes. Carlos fijó como regla que a Carpe se le gestiona con mucho cuidado en costos, mientras que a Flux se le puede incorporar lo que haga falta.

3.2 Costos de referencia

- Santiago estima unos 340 dólares por cada producción independiente. Carlos indicó que no debería pasar de 500 por cada una y que su estándar es no superar lo que cobraba Praxis por desarrollo: 800 dólares al mes.
- Alejo recordó que la propuesta de Advanced del 28 de octubre ronda los 900 dólares al mes y que la nueva propuesta debería quedar por debajo. Santiago señaló que esa propuesta contempla una sola base de datos, y Carlos pidió verificar la versión.

3.3 CI/CD y repositorio

- El participante E rechazó CodeCommit: por años dejó de aceptar nuevos proyectos y hubo dudas por su posible deprecación. Santiago explicó que AWS lo reactivó para todos los clientes y que su ventaja es el control centralizado de accesos.
- El participante F señaló que en GitHub ya se pueden proteger ramas y poner aprobadores. Carlos indicó que se necesita una licencia de 4 dólares por usuario (por 2 organizaciones, 8 dólares por persona) para dar permisos por equipo.
- Santiago aclaró que el repositorio es independiente de dónde se compile: se puede seguir con GitHub y cambiar luego a Azure DevOps u otro, un cambio de uno o dos días.
- Pablo mantuvo su posición: el repositorio solo para gestión de código y PR, y el resto en otro servicio que le quite la responsabilidad a las Actions. Se aclaró que con CodePipeline eso se cumple de forma autoadministrada, y que su preocupación por el costo y mantenimiento de Jenkins también se resuelve.
- Decisión: se aprueba por mayoría usar CodePipeline y CodeBuild en lugar de Jenkins, y seguir con GitHub en lugar de CodeCommit. Se pedirá a Advanced un alcance adicional.

3.4 Base de datos: Aurora y Postgres

- Pablo había propuesto Aurora para separar lectura y escritura con instancias stateless autoadministradas, útil en arquitecturas CQRS o muy transaccionales, junto con dos monolitos modulares (lectura y escritura).
- Santiago y el participante E lo ven sobreingeniería: Aurora cobra por operación, depende del ritmo de actualización de AWS y su valor aparece con cargas muy pesadas. Citaron casos de bancos y de un cliente anterior con 15 clientes en un Postgres pequeño. Pablo aceptó el punto, aclarando que se puede empezar con una sola base de datos.

Decisión: no se adopta Aurora; se continúa con RDS Postgres. El cambio de MariaDB a Postgres para Carpe quedó sin cerrar, porque Carlos pidió primero revisar el documento de arquitectura de Carpe y argumentar la conveniencia.

3.5 Datos, conciliador y BI

Carlos preguntó qué base de datos usar cuando el conciliador y la capa de BI consuman PDF, correos y XML, y se necesite un data warehouse o data lake para analítica y predicción. Pablo y Santiago descartaron Mongo. Santiago propuso mover la data histórica a S3 mediante AWS Glue, con Glacier para almacenamiento frío, y purgar la base transaccional. Como referencia, la base de Flux ocupa 1,5 GB desde mayo, y la preocupación llegaría cerca de 100 a 150 GB. Carlos dejó el tema como tarea de análisis.

3.6 Permisos en AWS

Santiago pidió un rol de administrador sobre la cuenta de AWS para configurar los roles del equipo (desarrolladores, DevOps y arquitecto), porque hoy solo hay acceso de lectura. Carlos dio el visto bueno y dará el go a Daniel de Advanced.



Acuerdos y próximos pasos

1. Santiago prepara la nueva propuesta con Flux y Carpe independientes en producción, y con desarrollo, calidad y producción.
2. Carlos pide a Daniel (Advanced) un alcance adicional con CodePipeline y CodeBuild en lugar de Jenkins, manteniendo GitHub.
3. Carlos autoriza a Daniel el rol de administrador sobre la cuenta de AWS.
4. Santiago revisa el documento de arquitectura de Carpe y argumenta el cambio de MariaDB a Postgres.
5. Definir el etiquetado de recursos por producto y ambiente, y revisar la zona de disponibilidad y la redundancia.
6. Ampliar la licencia de GitHub para permisos por equipo (4 dólares por usuario en cada una de las 2 organizaciones).
7. Analizar la base de datos para datos analíticos (warehouse o data lake) del conciliador y la capa de BI.

Temas pendientes

- Cambio de MariaDB a Postgres para Carpe.
- Costo final de la nueva arquitectura frente a los 340 dólares estimados y a los 500 de tope por producción.
- Verificar el alcance de la propuesta de Advanced del 28 de octubre (una o dos bases de datos).
- Esquema de redundancia y zonas de disponibilidad.
- Verificar con el equipo la asignación de nombres a las intervenciones.